

1) Después: direccionamiento IP

- **Servidor** = primera IP útil
- **Gateway del LAN** = última IP útil
- **Enlaces seriales** = subred /30

Prefijo	Máscara	Wildcard	Hosts	Salto (block size)
/24	255.255.255.0	0.0.0.255	254	256
/25	255.255.255.128	0.0.0.127	126	128
/26	255.255.255.192	0.0.0.63	62	64
/27	255.255.255.224	0.0.0.31	30	32
/28	255.255.255.240	0.0.0.15	14	16
/29	255.255.255.248	0.0.0.7	6	8
/30	255.255.255.252	0.0.0.3	2	4

2) Routing dinámico: OSPF

En tu práctica aparece OSPF en área 0, con router-id definido en cada router. Eso significa que la red está pensada para anunciarse por OSPF entre los routers.

Orden correcto para configurarlo:

1. `router ospf 1`
2. `router-id X.X.X.X`
3. `network ... area 0`
4. `passive-interface` en las LAN
5. dejar activos solo los enlaces entre routers

Ejemplo mental:

- interfaces hacia usuarios: **pasivas**
- seriales entre routers: **activas**

Comandos:

```
router ospf 1
```

```
router-id 1.1.1.1
network A.A.A.A W.W.W.W area 0
passive-interface fastethernet0/0
```

Verificación:

```
show ip ospf neighbor
show ip route
show ip protocols
```

6) Ruta por defecto

Se pone `default-information originate` si ese router es el punto de salida hacia “afuera” (internet).

```
ip route 0.0.0.0 0.0.0.0 <next-hop>
router ospf 1
default-information originate
```

Verificación:

```
show ip route
```

7) ACLs

a) ACL Estándar (solo origen)

Crear (numerada)

```
access-list 10 permit host 192.168.1.10
access-list 10 deny any
```

Crear (nombrada)

```
access-list standard ADMIN
    permit host 192.168.1.10
    deny any
```

Aplicar a interfaz

```
interface fa0/0
    ip access-group 10 in/out
```

IN → Filtra cuando el paquete ENTRA al router antes de que el router decida a dónde enviarlo. Usar cuando se quiere bloquear desde que llega

OUT → Filtra cuando el paquete SALE del router después de que ya fue enrutado

host 192.168.10.1 = solo esa IP

any = todas las IPs

¿Cómo se leen las reglas?

Caso 1: Permitir SOLO una IP

access-list 10 **permit host** 192.168.1.10 = **Permite SOLO** esa IP

access-list 10 **deny any** = **Niega TODO** lo demás

Caso 2: Bloquear SOLO una IP

access-list **deny host** 192.168.1.10 = **Bloquea SOLO** esa IP

access-list 10 **permit any** = **Permite TODO** lo demás

Caso 3: Permitir una RED completa

access-list 10 **permit** 192.168.1.0 0.0.0.255 = **Permite TODO** lo de esa red

access-list **deny any** = **Bloquea TODO** lo demás

Caso 4: Bloquear una RED completa

access-list 10 **deny** 192.168.1.0 0.0.0.255 = **Bloquea SOLO** esa red

access-list **permit any** = **Permite TODO** lo demás

Caso 5: Permitir VARIAS IPs específicas

access-list **permit host** 192.168.1.10

access-list **permit host** 192.168.1.20

access-list 10 **deny any**

Caso 6: Bloquear VARIAS IPs

access-list 10 **deny host** 192.168.1.10 = **Bloquea SOLO** estas IPs

access-list 10 **deny host** 192.168.1.20

access-list 10 **permit any** = **Permite TODO** lo demás

b) **ACL Extendida** (solo destino)

Crear (numerada)

access-list 120 **permit tcp any** 192.168.10.0 0.0.0.255 = **Permite SOLO** esa red

Crear (nombrada)

access-list extended ADMIN

permit tcp 192.168.10.0 0.0.0.255 **any** eq 80

Caso 1: Bloquear un servicio (ej. FTP)

access-list 100 **deny tcp any** host 10.0.0.1 eq ftp =

access-list 100 **permit** ip **any any** = Permitir TODO lo demás

Caso 2: Permitir solo web (HTTP)

access-list 100 **permit** tcp **any any** eq 80 = Permitir SOLO HTTP

access-list 100 deny ip any any

Caso 3: Permitir una red a un destino específico

access-list 100 permit **ip** 192.168.1.0 0.0.0.255 host 10.0.0.1

access-list 100 deny ip any any

Caso 4: Bloquear una red hacia un destino

access-list 100 deny ip 192.168.1.0 0.0.0.255 host 10.0.0.1

access-list 100 permit ip any any

a) ACL estándar para acceso administrativo SSH

Activar SSH

hostname NOMBRE

ip domain-name cisco.com

crypto key generate rsa

1024

username cisco secret cisco

line vty 0 4

login local

transport input ssh

Hacer conexión SSH...

ssh -l cisco 192.168.23.3

password:cisco

Crear ACL para acceso

ip access-list standard SSH

permit host 192.168.23.2

exit

line vty 0 4

access-class SSH in

VPN - General

#Instalar OpenVPN y easy-rsa
sudo apt install easy-rsa openvpn

#Crear directorio para certificados
Dentro de /etc/openvpn (o donde se te instale):
Make-cadir easy-rsa

#Crear directorio pki
Dentro de easy-rsa/:
./easyrsa init-pki

#Editar archivo de variables
Dentro del directorio de easy-rsa (o alrededores) podras encontrar el archivo:
vars
El cual contiene las variables con informacion de lo que se almacena en los certificados. Si se pide cambiarlos, desde ahi se editan.

#Crear certificado y clave de Autoridad de Certificacion (CA)
./easyrsa build-ca
IMPORTANTE: Recordar la frase solicitada, se requiere para firmar certificados posteriormente

#Crear claves (pública y privada) del servidor:
./easyrsa gen-req servidor-feiuv nopass
#nopass no pide contraseña, genera las claves en pki/private, pki/reqs

#Firmar certificado (clave publica del servidor)
./easyrsa sign-req server servidor-feiuv
#pedira la contraseña que pusiste en build-ca

#Crear claves para cliente (desde openvpn/easy-rsa)
./easyrsa gen-req cliente1-feiuv nopass
#pedira contraseña, ademas de confirmar nombre

#Firmar requerimiento del cliente:
./easyrsa sign-req client cliente1-feiuv

#Crear clave TLS CRYPT (para otra capa de seguridad)
Openvpn --genkey secret ta.key
#desde openvpn/server

```
#Hacer respaldo de la documentación
#Copiar /usr/share/doc/openvpn a /etc/openvpn/doc
#Copiar archivo de configuración de ejemplo desde la documentación, a
openvpn/server
cp openvpn/doc/examples/sample-config-files/server.conf openvpn/server
```

#ARCHIVO DE CONFIGURACION DE SERVIDOR - server.conf

#IMPORTANTE - HACER COPIA DE LOS ARCHIVOS ca.crt, servidor.crt, servidor.key, ta.key a el directorio openvpn/server. Revisa que en el archivo de configuración del servidor los nombres coincidan y se encuentren en el mismo directorio.

```
#EN EL ARCHIVO DE CONFIGURACIÓN - Deshabilitar Diffie-Hellman
;dh dh2048.pem
dh none
# EN EL ARCHIVO DE CONFIGURACIÓN - Habilitar topología de subred
Topology subnet
```

```
#Habilitar acceso a la red detrás del servidor (Validar IP)
push "route 192.168.1.0 255.255.255.0"
```

```
#Habilitar para que los clientes usen default gateway a través del túnel
push "redirect-gateway def1 bypass-dhcp"
```

```
#Asignar DNS a los clientes - cambia la ip si quieres cambiar DNS
push "dhcp-option DNS 208.67.222.222"
push "dhcp-option DNS 208.67.220.220"
```

```
#Habilitar clave de cifrado adicional
;tls-auth ta.key 0
tls-crypt ta.key 0
```

```
#Indicar algoritmo de cifrado
;cipher AES-256-CBC
cipher AES-256-GCM #OFRECE CIFRADO Y AUTENTICACION
auth SHA512
```

```
#Sin permisos de usuario y de grupo
;user openvpn
;group openvpn
user nobody
group nogroup
```

#ARCHIVO DE CONFIGURACION DE CLIENTE - cliente.conf

#Crear archivo de configuración del cliente

cp openvpn/doc/examples/sample-config-files/client.conf openvpn/client

IMPORTANTE: Verificar correspondencia de parámetros entre cliente y servidor

#Editar client.conf

#Agregar IP del servidor y puerto

remote 192.168.1.123 1194

#Sin permisos de usuario y de grupo

;user openvpn

;group openvpn

user nobody

group nogroup

#Comentar: esta información se incluirá en archive .ovpn del cliente

;ca ca.crt

;cert client.crt

;key client.key

;tls-auth ta.key 1

#Habilitar mismo cifrado que en el servidor

;cipher AES-256-CBC

cipher AES-256-GCM

auth SHA512

#REGLAS DE FIREWALL DE VPN

#Permitir reenvío de paquetes cada que se inicia el equipo:

#Editar archivo /etc/sysctl.conf (para configurar parámetros del kernel)

Quitar comentario a net.ipv4.ip_forward=1

Agregar "1" al archivo ip_forward (permite el reenvío en la sesión actual: echo 1 > /proc/sys/net/ipv4/ip_forward)

#Agregar reglas al firewall (iptables):

iptables -t nat -I POSTROUTING 1 -s 10.8.0.0/24 -o eth0 -j MASQUERADE (todo lo que entre por una ip privada saldrá por la IP "pública")

iptables -I INPUT 1 -i tun0 -j ACCEPT (acepta todo lo que entre por la interfaz virtual)

iptables -I FORWARD 1 -i eth0 -o tun0 -j ACCEPT (permite reenvío de la interfaz física a la virtual)

iptables -I FORWARD 1 -i tun0 -o eth0 -j ACCEPT (permite reenvío de la interfaz virtual a la física)

iptables -I INPUT 1 -i eth0 -p udp --dport 1194 -j ACCEPT (acepta todo lo que entre por la interfaz física, específicamente al puerto udp indicado)

#Hacer las reglas del firewall persistentes (aunque se reinicie el equipo)

instalar iptables-persistent

guardar reglas: netfilter-persistent save

#Configurar openvpn para que inicie en el arranque e iniciar servicio

systemctl -f enable openvpn-server@server.service

service openvpn-server@server start

#ARCHIVOS DE CLIENTE

#Archivo para conexión de cliente incluye:

Client.conf

Certificado de la Autoridad Certificadora (ca.crt)

Certificado del cliente (cliente.crt)

Clave privada cliente (cliente.key)

Clave privada tls-crypt (ta.key)

#Hacer copia de client.conf (mismo directorio), como plantilla.conf

Copiar en client/keys:

ca.crt

cliente.crt

cliente.key

ta.key

#Crear script para generar archivo .ovpn (make_config.sh)

#!/bin/bash

#Requiere argumento: Identificador del cliente

#

KEY_DIR=/etc/openvpn/client/keys

OUTPUT_DIR=/etc/openvpn/client/files

BASE_CONFIG=/etc/openvpn/client/plantilla.conf

```
cat ${BASE_CONFIG} \  
  <(echo -e '<ca>' \  
  ${KEY_DIR}/ca.crt \  
  <(echo -e '</ca>\n<cert>' \  
  ${KEY_DIR}/${1}.crt \  
  <(echo -e '</cert>\n<key>' \  
  ${KEY_DIR}/${1}.key \  
  <(echo -e '</key>\n<tls-crypt>' \  
  ${KEY_DIR}/ta.key \  
  <(echo -e '</tls-crypt>' \  
  > ${OUTPUT_DIR}/${1}.ovpn
```

#Conectarse al servidor OpenVPN

#Desde linux:

#Instalar Openvpn

```
sudo apt install easy-rsa openvpn
```

#Verificar que la IP que sale en el parametro "remote" del archivo .ovpn es la misma del servidor (dentro de la red virtual). Verificar tambien el puerto, en la misma linea debería estar.

#Conectarse/cargar el archivo .ovpn

```
sudo openvpn --config archivo.ovpn
```

#Verificar conexion

hacer ping a la ip del tunel del servidor (verificar con ip a, la interfaz tun0)

#Desde windows:

#Instala el cliente de openvpn

<https://openvpn.net/client/>

#Una vez instalado, simplemente ejecutalo y elige la opción que sale hasta abajo de la interfaz de importar archivo de configuración .ovpn

EIGRP

CONFIGURACIÓN BÁSICA EIGRP

enable

configure terminal

router eigrp 100 # 100 = AS (debe coincidir en todos los routers)

no auto-summary # MUY importante para redes modernas

network 192.168.1.0 0.0.0.255

network 10.0.0.0 0.0.0.3

👉 El **network** usa wildcard mask (inversa de la máscara normal)

VERIFICACIÓN

show ip protocols # Ver configuración EIGRP

show ip route # Ver rutas (las de EIGRP salen con "D")

show ip eigrp neighbors # Ver vecinos

show ip eigrp topology # Ver tabla de topología

CONFIGURAR INTERFACES (ANTES DE EIGRP)

interface fastEthernet 0/0

ip address 192.168.1.1 255.255.255.0

no shutdown

interface serial 0/0/0

ip address 10.0.0.1 255.255.255.252

clock rate 64000 # solo si es DCE

no shutdown

PASIVE INTERFACE (para no enviar updates)

router eigrp 100

passive-interface fastEthernet 0/0

👉 Útil para redes LAN donde no hay routers vecinos

REDISTRIBUCIÓN (básico)

router eigrp 100

redistribute static

MÉTRICA MANUAL

interface serial 0/0/0

bandwidth 1000
delay 2000

👉 EIGRP usa bandwidth + delay principalmente

🔒 AUTENTICACIÓN (básico)

```
interface serial 0/0/0  
ip authentication mode eigrp 100 md5  
ip authentication key-chain eigrp 100 CLAVES
```

```
key chain CLAVES  
key 1  
key-string 12345
```

🧹 BORRAR CONFIG

```
no router eigrp 100
```

⚡ TIP RÁPIDO (EXAMEN / PRÁCTICA)

- Todos los routers deben tener **mismo AS**
- Deben estar en la **misma red directamente conectada**
- Interfaces deben estar **up (no shutdown)**
- Usar siempre **no auto-summary**
- Verifica vecinos primero → si no hay vecinos, no hay rutas

OSPF

CONFIGURACIÓN BÁSICA

```
enable
configure terminal
router ospf 1
router-id 1.1.1.1
network 192.168.1.0 0.0.0.255 area 0
network 10.0.0.0 0.0.0.3 area 0
```

- ♦ **router ospf 1** → inicia OSPF (process-id local)
- ♦ **router-id** → identificador único (recomendado ponerlo manual)
- ♦ **network** → activa OSPF en interfaces que coincidan + asigna área

WILDCARD MASK (esto siempre cae)

Máscara	Wildcard
255.255.255.0	0.0.0.255
255.255.255.252	0.0.0.3
255.255.0.0	0.0.255.255
255.0.0.0	0.255.255.255

VERIFICACIÓN

```
show ip ospf neighbor
show ip route ospf
show ip protocols
show ip ospf
```

 Si no hay vecinos: revisa área, IP, máscara, conexión

INTERFAZ PASIVA (evita vecinos en LANs)

```
router ospf 1
passive-interface g0/1
```

DEFAULT ROUTE

```
ip route 0.0.0.0 0.0.0.0 192.168.1.1  
router ospf 1  
default-information originate
```

CAMBIAR COSTO (influye en la ruta)

```
interface g0/0  
ip ospf cost 10
```

PRIORIDAD (DR/BDR)

```
interface g0/0  
ip ospf priority 100
```

REINICIAR OSPF

```
clear ip ospf process
```

ÁREA STUB (básico)

```
router ospf 1  
area 1 stub
```

DEBUG

```
debug ip ospf adj  
debug ip ospf events
```

MINI CHECKLIST (cuando no jala)

- **no shutdown** en interfaces
- IPs correctas
- Wildcard bien calculado
- Misma área
- Router-ID único
- ¿Responden ping?

Todos deben estar conectados directamente para ospf, como eigrp.

RIP

 Activar RIP (modo básico)

```
enable
configure terminal
router rip
version 2
no auto-summary
```

 Anunciar redes

(se hace con networks, no en interfaz directamente)

```
network 192.168.1.0
network 10.0.0.0
```

 Usa redes “clase A/B/C”, RIP se encarga del resto.

 Verificar configuración

```
show ip protocols
show ip route
```

 Busca rutas con **R** → eso es RIP trabajando.

 Ver vecinos / actualizaciones

```
debug ip rip
```

 Esto spamea bastante, úsalo con cuidado:

```
undebug all
```

 Interfaces pasivas (no enviar updates por ahí)

```
router rip
passive-interface fastEthernet 0/0
```

 Métrica (distancia = saltos)

RIP usa **hop count**:

- Máximo: 15 saltos
 - 16 =  inalcanzable
-

Config útil extra

Forzar envío por interfaz específica

```
router rip
```

```
network 192.168.1.0
```

(RIP activará automáticamente las interfaces que coincidan con esa red)

Redistribución (básico)

```
router rip
```

```
redistribute static
```

Comandos rápidos de diagnóstico

```
show ip route rip
```

```
show running-config
```

```
ping [ip]
```

```
traceroute [ip]
```

Cosas importantes que muchos olvidan

- Siempre usa **version 2** (RIPv1 está viejo y limitado)
- **no auto-summary** evita errores entre redes discontinuas
- **RIP sí necesita que las redes estén directamente conectadas** (igual que EIGRP)
- No configuras por interfaz como OSPF, todo es con **network**

Lo de las VLAN

1) Crear / verificar la VLAN

```
conf t
```

```
vlan 10
```

```
name USERS
```

```
exit
```

 Si ya existe, no pasa nada, solo la aseguras

2) Configurar la SVI (la “IP de la VLAN”)

```
interface vlan 10
```

```
ip address 192.168.10.1 255.255.255.0
no shutdown
exit
```

Aquí es donde muchos fallan

Esto **NO basta** para que haga ping

3) Asignar puertos a la VLAN (CLAVE)

```
interface fa0/1
switchport mode access
switchport access vlan 10
no shutdown
```

```
interface fa0/2
switchport mode access
switchport access vlan 10
no shutdown
```

👉 Debe haber al menos **un puerto activo en esa VLAN**

4) Verificar que la VLAN esté arriba

```
show ip interface brief
```

👉 Debes ver algo como:

```
Vlan10  192.168.10.1  up  up
```

5) (Si no responde) revisar puertos

```
show vlan brief
```

👉 Verifica que los puertos sí estén en VLAN 10

6) (Si necesitas comunicación entre redes)

```
ip default-gateway 192.168.10.254
```

👉 Solo si el switch es capa 2

7) Configurar SSH

```
hostname Switch1
```

```
ip domain-name red.local
```

```
crypto key generate rsa
```

```
username admin secret 1234
```

```
line vty 0 4
```

```
login local
```

```
transport input ssh
```

8) Restringir SSH a UNA PC

```
ip access-list standard ADMIN
```

```
permit host 192.168.10.10
```

```
deny any
```

```
line vty 0 4
```

```
access-class ADMIN in
```

9) ACL para bloquear acceso web (ejemplo)

```
access-list 100 deny tcp any host 192.168.10.100 eq 80
```

```
access-list 100 permit ip any any
```

Aplicarla:

```
interface fa0/1
```

```
ip access-group 100 in
```



1. Preparación del servidor

◆ Instalar paquetes

```
sudo apt update  
sudo apt install openvpn easy-rsa -y
```

◆ Crear entorno PKI

```
cd /etc/openvpn  
sudo make-cadir easy-rsa  
cd easy-rsa
```

Inicializar:

```
./easyrsa init-pki
```



2. Autoridad Certificadora (CA)

```
./easyrsa build-ca
```

👉 Te pedirá una contraseña (CA passphrase)

Se generan:

- `pki/ca.crt`
 - `pki/private/ca.key`
-



3. Certificado del servidor

♦ Crear solicitud

```
./easyrsa gen-req servidor nopass
```

♦ Firmar

```
./easyrsa sign-req server servidor
```



4. Certificado del cliente

```
./easyrsa gen-req cliente1 nopass
```

```
./easyrsa sign-req client cliente1
```



5. Clave TLS (seguridad extra)

```
cd /etc/openvpn/server
```

```
openvpn --genkey secret ta.key
```



6. Configuración del servidor

♦ Copiar archivo base

```
sudo cp /usr/share/doc/openvpn/examples/sample-config-files/server.conf /etc/openvpn/server/  
cd /etc/openvpn/server
```

♦ Copiar certificados necesarios

```
sudo cp /etc/openvpn/easy-rsa/pki/ca.crt .  
sudo cp /etc/openvpn/easy-rsa/pki/issued/servidor.crt .  
sudo cp /etc/openvpn/easy-rsa/pki/private/servidor.key .  
sudo cp /etc/openvpn/server/ta.key .
```

◆ Editar **server.conf**

```
sudo nano server.conf
```

Cambios clave:

```
ca ca.crt  
cert servidor.crt  
key servidor.key
```

```
dh none  
topology subnet
```

```
server 10.8.0.0 255.255.255.0
```

```
push "route 192.168.1.0 255.255.255.0"  
push "redirect-gateway def1 bypass-dhcp"
```

```
push "dhcp-option DNS 8.8.8.8"  
push "dhcp-option DNS 1.1.1.1"
```

```
tls-crypt ta.key
```

```
cipher AES-256-GCM  
auth SHA512
```

```
user nobody  
group nogroup
```

7. Configuración de red (MUY IMPORTANTE)

◆ Activar IP forwarding

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

Persistente:

```
sudo nano /etc/sysctl.conf
```

```
net.ipv4.ip_forward=1
```

Aplicar:

```
sudo sysctl -p
```

◆ Configurar iptables (NAT)

👉 Sustituye `enp1s0` por tu interfaz real

```
iptables -t nat -A POSTROUTING -s 10.8.0.0/24 -o enp1s0 -j MASQUERADE
```

```
iptables -A INPUT -i tun0 -j ACCEPT
```

```
iptables -A FORWARD -i enp1s0 -o tun0 -j ACCEPT
```

```
iptables -A FORWARD -i tun0 -o enp1s0 -j ACCEPT
```

```
iptables -A INPUT -i enp1s0 -p udp --dport 1194 -j ACCEPT
```

◆ Guardar reglas

```
sudo apt install iptables-persistent -y
```

```
sudo netfilter-persistent save
```

8. Iniciar OpenVPN

```
sudo systemctl enable openvpn-server@server
```

```
sudo systemctl start openvpn-server@server
```

Verificar:

```
systemctl status openvpn-server@server
```

9. Configuración del cliente

◆ Copiar plantilla

```
mkdir -p ~/client-configs/files  
cp /usr/share/doc/openvpn/examples/sample-config-files/client.conf ~/client-configs/base.conf
```

◆ Editar **base.conf**

```
nano ~/client-configs/base.conf
```

Cambiar:

```
remote IP_DEL_SERVIDOR 1194
```

```
user nobody  
group nogroup
```

```
cipher AES-256-GCM  
auth SHA512
```

comentar estas líneas:

```
;ca ca.crt  
;cert client.crt  
;key client.key  
;tls-auth ta.key 1
```

Agregar:

```
key-direction 1
```



10. Preparar archivos del cliente

```
mkdir -p ~/client-configs/keys
```

Copiar:

```
cp /etc/openssl/easy-rsa/pki/ca.crt ~/client-configs/keys/  
cp /etc/openssl/easy-rsa/pki/issued/cliente1.crt ~/client-configs/keys/  
cp /etc/openssl/easy-rsa/pki/private/cliente1.key ~/client-configs/keys/  
cp /etc/openssl/server/ta.key ~/client-configs/keys/
```



11. Script para generar .ovpn

```
nano ~/client-configs/make_config.sh
```

Contenido:

```
#!/bin/bash
```

```
KEY_DIR=~/client-configs/keys  
OUTPUT_DIR=~/client-configs/files  
BASE_CONFIG=~/client-configs/base.conf
```

```
cat ${BASE_CONFIG} \  
  <(echo -e '<ca>') \  
  ${KEY_DIR}/ca.crt \  
  <(echo -e '</ca>\n<cert>') \  
  ${KEY_DIR}/cliente1.crt \  
  <(echo -e '</cert>\n<key>') \  
  ${KEY_DIR}/cliente1.key \  
  <(echo -e '</key>\n<tls-crypt>') \  
  ${KEY_DIR}/ta.key \  
  <(echo -e '</tls-crypt>') \  
  > ${OUTPUT_DIR}/cliente1.ovpn
```

Permisos:

```
chmod +x ~/client-configs/make_config.sh
```

◆ Generar cliente

```
~/client-configs/make_config.sh cliente1
```



12. Conectar cliente

Linux:

```
sudo openvpn --config cliente1.ovpn
```

Windows:

- Importar **.ovpn** en OpenVPN Connect
 - Conectar
-



13. Pruebas finales

Cliente:

```
ping 8.8.8.8  
curl ifconfig.me  
tracert 8.8.8.8 # Windows  
tracert 8.8.8.8 # Linux
```

👉 Debe iniciar en:

10.8.0.1

Servidor:

```
iptables -t nat -L -n -v
```

👉 Ver tráfico en MASQUERADE